Reinforcement Learning

Lecture 2: Imitation Learning

SJTU

Outline

1. From Supervised Learning to Imitation Learning
2. Behavioral Cloning
3. Does Behavioral Cloning Work?
4. Theory: How Bad is Distributional Shift?
5. Beyond Behavioral Cloning
6. Handling Complex Behaviors
7. Case Studies
8. Summary

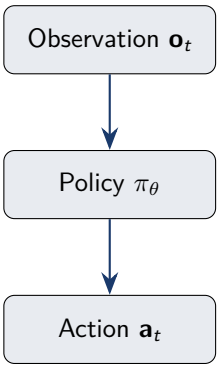
Recap: Sequential Decision Making

- Last lecture: RL is about agents making **sequential decisions**
- Today: Can we learn decision-making from **demonstrations**?
- Key idea: given expert behavior, learn a policy via **supervised learning**

Imitation Learning: learn to act by observing an expert

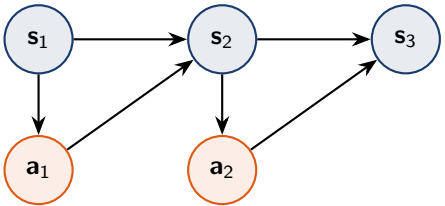
Terminology and Notation

- $\mathbf{o}_t$ — **observation** at time t
  - What the agent perceives (e.g., camera image)
- $\mathbf{s}_t$ — **state** at time t
  - True underlying state of the world
- $\mathbf{a}_t$ — **action** at time t
  - What the agent does (e.g., motor command)
- $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ — **policy**
  - Maps observations to actions
  - θ = learnable parameters



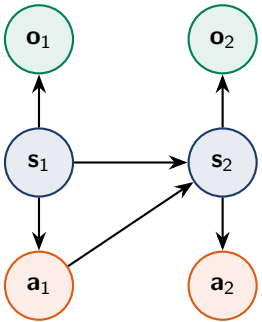
State vs. Observation

Fully Observed (MDP)



- Policy: $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$
- State is fully known

Partially Observed (POMDP)



- Policy: $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$
- State hidden; only observations

The Markov Property

Markov Property

$$p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) = p(\mathbf{s}_{t+1}|\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_t, \mathbf{a}_t)$$

- The **state** is a sufficient summary of the past
- Future is conditionally independent of history given current state
- Observations generally do **not** satisfy the Markov property
  - A single camera image may not capture all relevant information
  - Example: cheetah image — cannot tell if running left or right
- Implication: policies over observations may need **history**

Imitation Learning Setup

- **Given:** Expert demonstrations $\{(\mathbf{o}_1^{(i)}, \mathbf{a}_1^{(i)}, \ldots, \mathbf{o}_H^{(i)}, \mathbf{a}_H^{(i)})\}_{i=1}^N$
- **Goal:** Learn a policy π_θ that behaves like the expert
- Simplest approach: treat it as **supervised learning**!

Behavioral Cloning: directly regress actions from observations

Behavioral Cloning Algorithm

Algorithm: Behavioral Cloning

1. Collect demonstrations from expert: $\mathcal{D} = \{(\mathbf{o}_t^{(i)}, \mathbf{a}_t^{(i)})\}$
2. Train policy π_θ via **maximum likelihood**:

$$\theta^* = \arg\max_\theta \sum_{i=1}^{N} \sum_{t=1}^{H} \log \pi_\theta\left(\mathbf{a}_t^{(i)} \mid \mathbf{o}_t^{(i)}\right)$$

3. Deploy π_{θ^*} in the environment

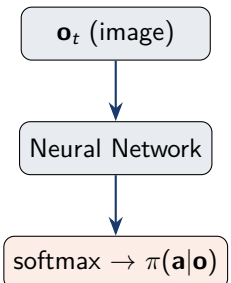
- This is just supervised learning with $\mathbf{x} = \mathbf{o}_t$, $\mathbf{y} = \mathbf{a}_t$
- Ignores temporal structure — treats each $(\mathbf{o}_t, \mathbf{a}_t)$ independently

Discrete Actions: Classification

- Actions from a finite set: $\mathbf{a}_t \in \{1, 2, \ldots, K\}$
- Policy outputs **logits** over actions:

$$\pi_\theta(\mathbf{a}_t = k \mid \mathbf{o}_t) = \frac{\exp(f_k(\mathbf{o}_t))}{\sum_{j=1}^{K} \exp(f_j(\mathbf{o}_t))}$$

- Loss: cross-entropy (negative log-likelihood)
- Example: go left, go right, go straight

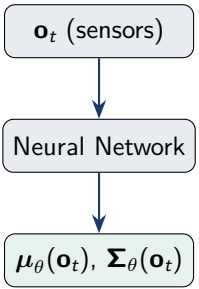


Continuous Actions: Regression

- Actions are continuous: $\mathbf{a}_t \in \mathbb{R}^d$
- Model as Gaussian distribution:

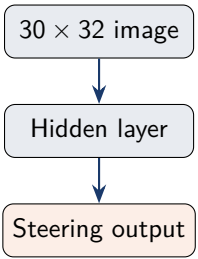
$$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t) = \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{o}_t), \boldsymbol{\Sigma}_\theta(\mathbf{o}_t))$$

- Network outputs mean $\boldsymbol{\mu}$ (and optionally variance $\boldsymbol{\Sigma}$)
- Loss: mean squared error (for fixed variance)
- Example: steering angle, joint torques



Historical Example: ALVINN (1989)

- One of the **first** imitation learning systems
- Autonomous driving at Carnegie Mellon
- Single-layer neural network
- Input: 30×32 pixel camera image
- Output: steering direction (discretized)
- Drove at up to 70 mph on highways
- Remarkably effective for such a simple model!



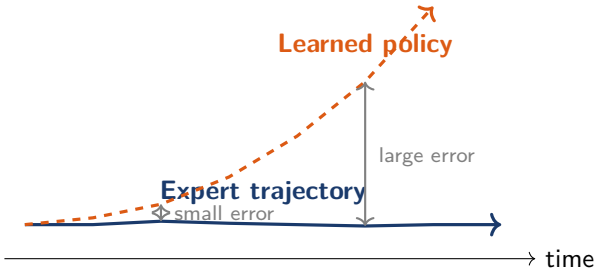
Pomerleau, 1989

The Problem with Behavioral Cloning

Distributional Shift: training and test distributions differ!

- Training data comes from **expert's** state distribution: $p_{\text{data}}(\mathbf{o}_t)$
- At test time, the **learned policy** visits different states: $p_{\pi_\theta}(\mathbf{o}_t)$
- Small errors accumulate $\rightarrow$ policy drifts into unfamiliar states
- In unfamiliar states, errors grow even larger $\rightarrow$ compounding errors

Distributional Shift: Intuition



- At each step, the policy makes a **small** mistake
- Mistakes push it into states **not seen in training**
- Errors compound over time $\rightarrow$ trajectory diverges

When Does Behavioral Cloning Work?

Easier settings:

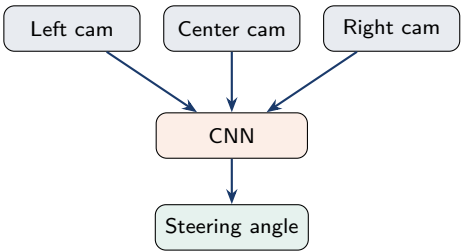
- Task is **short horizon** (few steps)
- Environment is **stable/forgiving**
  - Small errors don't cascade
- **Lots of diverse data** available
- Very **expressive model** (large neural net)

Harder settings:

- Task is **long horizon**
- Environment is **unstable**
  - Small errors lead to catastrophe
- **Limited demonstrations**
- **Multimodal** expert behavior

Success Story: NVIDIA Self-Driving (Bojarski et al., 2016)

- End-to-end behavioral cloning for **lane following**
- CNN maps camera images $\rightarrow$ steering angle
- Key trick: **data augmentation** with side cameras
  - Left camera $\rightarrow$ steer right
  - Right camera $\rightarrow$ steer left
  - Simulates **recovery** from mistakes
- Effectively teaches the policy **how to correct**
- Stabilizes behavioral cloning!



Data Augmentation as Implicit DAgger

- Side cameras simulate being **off the expert trajectory**
- Left camera image $\approx$ "you drifted right; steer left"
- This provides supervision for **recovery behavior**
- Equivalent to augmenting the training distribution:

$$p_{\text{augmented}}(\mathbf{o}_t) \approx p_{\pi_\theta}(\mathbf{o}_t)$$

- General principle: if you can augment data to cover likely mistakes, BC can work well

Powerful Models Can Help

- With enough data and a **sufficiently expressive** model:
  - Model can learn to generalize to unseen states
  - Effectively reduces distributional shift
- Modern large-scale BC works surprisingly well:
  - **Multi-task learning** across many environments
  - **Large diverse datasets** covering many situations
  - Transformer-based architectures
- But: still fundamentally limited by distributional shift for long horizons

Formalizing the Problem

- Define a **cost function**: $c(\mathbf{s}_t, \mathbf{a}_t) = \begin{cases} 0 & \text{if } \mathbf{a}_t = \pi^*(\mathbf{s}_t) \\ 1 & \text{otherwise} \end{cases}$
- Assume the learned policy makes mistakes with probability ϵ:

$$\pi_\theta(\mathbf{a} \neq \pi^*(\mathbf{s}) \mid \mathbf{s}) \leq \epsilon \quad \text{for all } \mathbf{s} \sim p_{\text{train}}(\mathbf{s})$$

- **Question:** What is the expected total cost over a trajectory of length H?

$$\mathbb{E}\left[\sum_{t=1}^{H} c(\mathbf{s}_t, \mathbf{a}_t)\right] = ?$$

Naïve Bound vs. Reality

Naïve (i.i.d.) Bound

If data were i.i.d.:

$$\mathbb{E}\left[\sum_{t=1}^{H} c_t\right] \leq \epsilon H$$

Linear in horizon — not bad!

Actual Bound (Ross et al., 2011)

Due to distributional shift:

$$\mathbb{E}\left[\sum_{t=1}^{H} c_t\right] \leq \epsilon H^2$$

Quadratic in horizon — much worse!

- The H^2 factor is the price of **compounding errors**
- For $H = 1000$ and $\epsilon = 0.01$: i.i.d. gives 10, actual gives 10,000!

Deriving the Bound: State Distribution Under π_θ

- At each step, with probability $(1-\epsilon)$, the policy acts like the expert
- The state distribution under π_θ can be decomposed:

$$p_{\pi_\theta}(\mathbf{s}_t) = (1-\epsilon)^t \, p_{\text{train}}(\mathbf{s}_t) + \left(1-(1-\epsilon)^t\right) p_{\text{mistake}}(\mathbf{s}_t)$$

- $p_{\text{train}}(\mathbf{s}_t)$: distribution if no mistakes were made
- $p_{\text{mistake}}(\mathbf{s}_t)$: some (possibly bad) distribution after a mistake
- As t grows, $(1-\epsilon)^t \to 0$ — we inevitably drift away

Total Variation Divergence Bound

Key Result

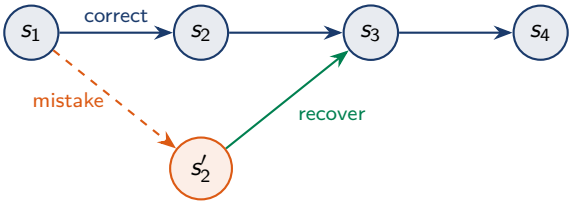
Using total variation divergence:

$$|p_{\pi_\theta}(\mathbf{s}_t) - p_{\text{train}}(\mathbf{s}_t)| \leq 2\epsilon t$$

for all time steps t.

- The "useful" bound: $\sum_{t=1}^{H} |p_{\pi_\theta}(\mathbf{s}_t) - p_{\text{train}}(\mathbf{s}_t)| \leq 2\epsilon \cdot \frac{H(H-1)}{2} = O(\epsilon H^2)$
- This is **tight**: there exist MDPs where the quadratic bound is achieved
- The recovery paradox: even if mistakes are "recoverable," compounding still happens because recovery itself visits non-training states

The Recovery Paradox



- Even if the policy can **recover** from s'_2 back to s_3...
- The cost at s'_2 is **still incurred** (we made a mistake)
- The recovered state s_3 may differ subtly from the expert's s_3
- Recovery does not eliminate compounding — it just changes the constant

Can We Do Better?

- Behavioral cloning suffers from $O(\epsilon H^2)$ cost
- **Goal:** achieve $O(\epsilon H)$ — linear cost like the i.i.d. case
- Key idea: collect training data from $p_{\pi_\theta}(\mathbf{o}_t)$ instead of $p_{\text{data}}(\mathbf{o}_t)$
- But we need the **expert's labels** for those observations!

DAgger: Dataset Aggregation (Ross et al., 2011)

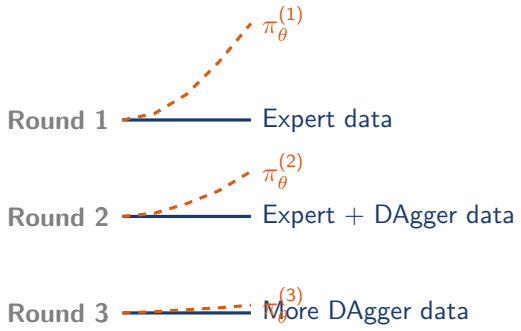
DAgger: Dataset Aggregation

Algorithm: DAgger

1. Train π_θ on expert data $\mathcal{D} = \{(\mathbf{o}_t, \mathbf{a}_t)\}$ (behavioral cloning)
2. **Run** π_θ to collect observations $\{\mathbf{o}_1, \ldots, \mathbf{o}_M\}$
3. **Ask expert** to label these observations: $\mathbf{a}_t^* = \pi^*(\mathbf{o}_t)$
4. **Aggregate**: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{o}_t, \mathbf{a}_t^*)\}$
5. Go to step 1

- Now training data covers states the **learned policy actually visits**
- Converges to $O(\epsilon H)$ bound — no more compounding!
- Downside: requires interactive access to the expert

DAgger: Why It Works



- Each round, the training distribution better matches the policy's distribution
- After enough rounds: $p_{\text{train}}(\mathbf{o}_t) \approx p_{\pi_\theta}(\mathbf{o}_t)$

DAgger: Practical Considerations

Advantages:

- Provably optimal bounds
- Simple to implement
- Works well in practice
- Can use any supervised learner

Challenges:

- Needs **interactive expert**
- Expert must label **on-policy** states
- For robotics: human must watch and provide corrections in real time
- Expensive and sometimes unsafe

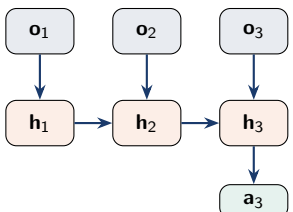
Non-Markovian Behavior

- Standard BC: $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ — only uses **current** observation
- But observations are often **not Markov**!
  - A single image doesn't tell velocity, acceleration, etc.
  - Expert behavior may depend on what happened **before**
- Solution: use **history-dependent** policies:

$$\pi_\theta(\mathbf{a}_t|\mathbf{o}_1, \mathbf{o}_2, \ldots, \mathbf{o}_t)$$

Sequence Models for Imitation

- **RNN / LSTM**: process observation sequence, maintain hidden state
- **Transformer**: attention over full history
  - Can capture long-range dependencies
  - Scales well with modern architectures
- Both produce $\pi_\theta(\mathbf{a}_t|\mathbf{o}_{1:t})$
- Particularly important for POMDP settings

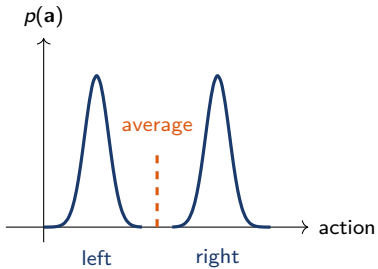


Causal Confusion

- With history-dependent policies, a subtle problem arises:
- **Causal confusion** (de Haan et al., 2019):
  - Policy learns to use **past actions** as a shortcut
  - Instead of understanding the environment, it copies its own past behavior
  - Example: "I braked before $\rightarrow$ I should keep braking"
- The policy uses spurious correlations in the **history**
- Mitigation: careful feature selection, causal reasoning

Multimodal Behavior

- Expert demonstrations may be **multimodal**:
  - Obstacle ahead: go left **or** go right
  - Both are correct, but **averaging** is wrong!
- Single Gaussian $\rightarrow$ averages modes $\rightarrow$ crashes
- Need **expressive action distributions**



Solutions for Multimodal Actions

1. **Mixture of Gaussians:**

$$\pi_\theta(\mathbf{a}|\mathbf{o}) = \sum_{k=1}^{K} w_k \mathcal{N}(\boldsymbol{\mu}_k(\mathbf{o}), \boldsymbol{\Sigma}_k(\mathbf{o}))$$

   Simple, but need to choose K; doesn't scale to high dimensions

2. **Latent Variable Models (e.g., cVAE):**

$$\pi_\theta(\mathbf{a}|\mathbf{o}) = \int p_\theta(\mathbf{a}|\mathbf{o}, \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$$

   Sample $\mathbf{z} \sim \mathcal{N}(0, I)$, then decode to action

3. **Diffusion Models:** Iteratively denoise $\mathbf{a}^{(K)} \sim \mathcal{N}(0, I) \rightarrow \mathbf{a}^{(0)}$
   - Highly expressive; state-of-the-art in robotics

Autoregressive Discretization

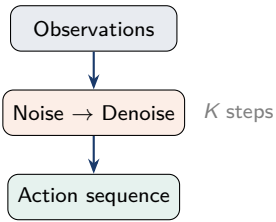
- Alternative: discretize action space and predict **one dimension at a time**
- Factor the joint distribution:

$$\pi_\theta(\mathbf{a}|\mathbf{o}) = \prod_{d=1}^{D} \pi_\theta(a_d|\mathbf{o}, a_1, \ldots, a_{d-1})$$

- Each factor is a **categorical distribution** over bins
- Naturally handles multimodality without Gaussian assumptions
- Used in modern robotics systems (e.g., RT-1)

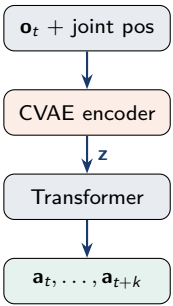
Diffusion Policy (Chi et al., 2023)

- Uses **diffusion models** to generate robot actions
- Predicts a **sequence** of future actions (action chunk)
- Conditions on observation history via cross-attention
- Naturally multimodal: different denoising paths $\rightarrow$ different behaviors
- State-of-the-art on multiple manipulation benchmarks



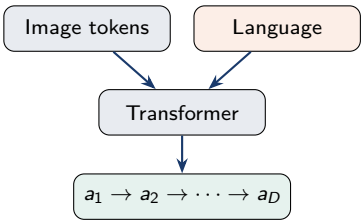
ACT: Action Chunking with Transformers (Zhao et al., 2023)

- **CVAE** + **Transformer** architecture
- Predicts chunks of k future actions at once
- Latent variable **z** captures style/mode of behavior
- Temporal ensembling: overlapping chunks for smooth execution
- Enables low-cost bimanual robot manipulation



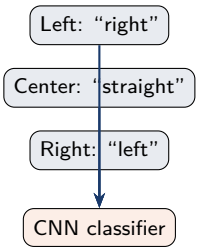
RT-1: Robotics Transformer (Brohan et al., 2023)

- Large-scale behavioral cloning for **real robots**
- Trained on 130k+ demonstrations across 700+ tasks
- **Autoregressive discretization** of actions
- Tokenizes images with EfficientNet, actions with discrete bins
- Transformer decoder generates action tokens sequentially
- Demonstrates power of **scale + diverse data**



Trail Following (Giusti et al., 2016)

- Autonomous trail following with a **monocular camera**
- Three classes: go left, go straight, go right
- Training trick: **three cameras** (like NVIDIA)
  - Head-mounted: center camera
  - Left/right: serve as augmentation
- Simple CNN classifier works well
- Example of how **clever data collection** mitigates distributional shift



Summary

- **Imitation learning**: learn policies from expert demonstrations
- **Behavioral cloning**: simple supervised learning approach
  - Works for short horizons or with data augmentation
  - Suffers from distributional shift: $O(\epsilon H^2)$ error bound
- **DAgger**: iteratively collects on-policy data with expert labels
  - Achieves $O(\epsilon H)$ bound, but needs interactive expert
- **Handling complexity**:
  - Non-Markovian $\rightarrow$ sequence models (RNN, Transformer)
  - Multimodal $\rightarrow$ mixture models, cVAE, diffusion, autoregressive
- Modern systems (Diffusion Policy, ACT, RT-1) combine these ideas at scale

Next lecture: Introduction to Reinforcement Learning — learning from **rewards** instead of demonstrations

Thank You

Questions?